

Distributed Voting System

A Multi-Process, Multi-Region Electronic Voting System in C

Project Report

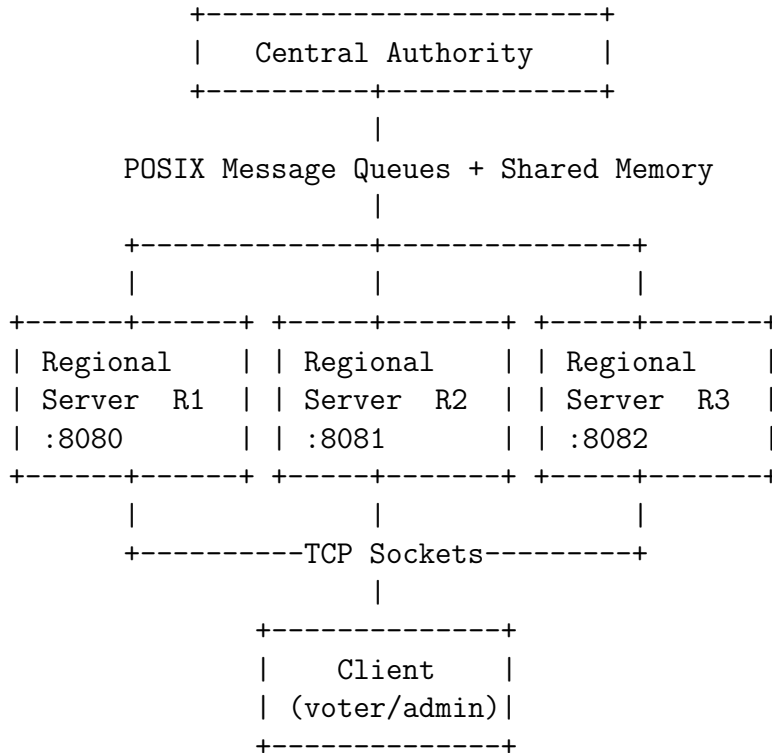
Shashank Peddi

Roll Number: BT2024210

April 30, 2026

1 System Architecture

The overall communication topology is shown below:



2 Implementation of Required Concepts

2.1 POSIX Message Queues

Two message queues are created by the Central Authority at startup using `msgget` with IPC keys `KEY_REQ` (1234) and `KEY_RES` (5678). They form the primary IPC channel between the Authority and every Regional Server.

How they are used:

- **Server registration:** On startup, each server sends a `CONNECT` message (with its PID and region) on `KEY_REQ` and waits for the Authority's acknowledgement on `KEY_RES`.
- **Global duplicate-vote check:** When a voter casts a vote, the server sends a `CHECK` message containing the voter ID to the Authority, which checks and updates the global voted file, then replies with `status = 1` (allowed) or `0` (already voted).
- **Audit verification and result collection:** The Authority sends `VERIFY` or `RESULTS` messages addressed to specific server PIDs. The server's background thread receives these and responds accordingly.

The shared message struct is defined in `common/utils.h`:

Listing 1: Message struct used on both queues

```
1 struct msg {
```

```

2  long type;           /* message queue routing field */
3  int  sender_pid;
4  char cmd[20];        /* "CONNECT", "CHECK", "VERIFY", "RESULTS
   " */
5  char voter_id[20];
6  char candidate;
7  int  status;
8  char region[4];
9  };

```

The `type` field acts as a routing key: requests to a specific server use `type = server_pid`; replies back to the Authority use `type = 1` (the Authority always uses `sender_pid = 1`).

2.2 POSIX Shared Memory

Shared memory is used in two places:

Server Registry (SHM_KEY)

The Central Authority creates a shared memory segment that stores a list of registered servers and the total number of connected servers. This allows all Regional Servers to attach to the same memory and confirm registration, while the Authority can track active servers and manage them.

Listing 2: Shared memory registry structure

```

1  typedef struct
2  {
3      pid_t pid;
4      char region[4];
5      int connected;
6  } server_info;
7
8  static server_info *servers;
9  static int *server_count_ptr;
10 static volatile int keep_running = 1;
11 static int req_q, res_q;

```

This shared memory acts like a central live registry that all processes can access, ensuring that server status is visible system-wide without repeated message passing.

Result Collection (SHM_RESULT_KEY)

When the Authority triggers result collection, it allocates a `result_shm` segment pre-populated with one slot per registered server:

Listing 3: Shared memory layout for result collection

```

1  typedef struct {
2      pid_t  server_pid;
3      char   region[4];
4      int    ready;           /* 0=pending, 1=done, -1=error */
5      int    total_votes;

```

```

6     int     counts[MAX_CANDIDATES];
7 } result_entry;
8
9 typedef struct {
10     int     server_count;
11     result_entry entries[MAX_SERVERS];
12 } result_shm;

```

Each server locates its own slot by PID, writes its tally data, and sets `ready = 1`. The Authority polls until all slots are ready (up to 5 seconds), then aggregates and displays the totals.

2.3 UNIX Signals

Signals are the lightweight, real-time control channel from the Authority to Regional Servers:

Signal	Sent by	Effect on server
SIGUSR1	Authority	Sets <code>vote_flag = 1</code> (voting enabled)
SIGUSR2	Authority	Sets <code>vote_flag = 0</code> (voting disabled)
SIGTERM	Authority	Sets <code>server_running = 0</code> , calls <code>_exit(0)</code>

Table 1: Signals used in the system

The server registers its signal handlers with `signal()` and protects the `vote_flag` variable with a `pthread_mutex` to avoid data races between the signal handler and voting threads:

Listing 4: Thread-safe `vote_flag` access

```

1 static volatile sig_atomic_t vote_flag = 0;
2 static pthread_mutex_t vote_flag_mutex = PTHREAD_MUTEX_INITIALIZER;
3
4 int get_vote_flag(void) {
5     pthread_mutex_lock(&vote_flag_mutex);
6     int v = vote_flag;
7     pthread_mutex_unlock(&vote_flag_mutex);
8     return v;
9 }

```

The Authority can target all regions at once (`send_signal_to_all_servers`) or a single region (`send_signal_to_region`) through menu options 1–4.

2.4 TCP Sockets

Each Regional Server creates a TCP listening socket bound to a fixed port (R1: 8080, R2: 8081, R3: 8082):

Listing 5: Server socket setup

```

1 int server_fd = socket(AF_INET, SOCK_STREAM, 0);

```

```

2  setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)
   );
3  addr.sin_family      = AF_INET;
4  addr.sin_port        = htons(port);
5  addr.sin_addr.s_addr = INADDR_ANY;
6  bind(server_fd, (struct sockaddr *)&addr, sizeof(addr));
7  listen(server_fd, 10);

```

The client uses `connect()` to reach a server, then exchanges text commands:

- **LOGIN** <voter_id> → server returns a signed token or **INVALID VOTER**
- **VOTE** <token> <candidate> → server returns **SUCCESS** or **FAILURE**
- **ADMIN_AUTH** <hash> → server returns **ADMIN_OK** or **ADMIN_FAIL**
- **ADMIN_ENABLE** / **ADMIN_DISABLE** / **ADMIN_VERIFY** / **ADMIN_EXIT** → admin session commands

2.5 POSIX Threads (pthreads)

Two uses of pthreads:

1. **Authority listener thread:** Each Regional Server spawns a detached background thread (`listen_authority`) that blocks on `msgrcv` waiting for **VERIFY** or **RESULTS** commands from the Authority, keeping the main accept loop unblocked.
2. **Per-client threads:** For every accepted TCP connection the server spawns a new detached thread running `handle_client`, enabling concurrent service of multiple voters:

Listing 6: Spawning a thread per client connection

```

1  int *client_fd = malloc(sizeof(int));
2  *client_fd = accept(server_fd, NULL, NULL);
3  pthread_t tid;
4  pthread_create(&tid, NULL, handle_client, client_fd);
5  pthread_detach(tid);

```

2.6 fork()

The Central Authority uses `fork()` to split into two concurrent activities after authentication:

- **Child process** — runs `handle_user_input()`, presenting the interactive menu and dispatching commands.
- **Parent process** — runs the IPC message-receive loop, handling **CONNECT** and **CHECK** requests from Regional Servers with `IPC_NOWAIT` polling. It calls `waitpid(..., WNOHANG)` to detect when the child exits, then cleans up IPC resources and exits.

This design keeps the UI responsive while the Authority handles IPC traffic without blocking either activity.

2.7 File I/O with fcntl Locking

All persistent data files are protected by POSIX advisory locks (fcntl) implemented in common/utils.c:

Listing 7: POSIX write lock and unlock

```

1 void lock_file(int fd) {
2     struct flock lock = { F_WRLCK, SEEK_SET, 0, 0, 0 };
3     fcntl(fd, F_SETLK, &lock);    /* blocking write lock */
4 }
5 void read_lock_file(int fd) {
6     struct flock lock = { F_RDLCK, SEEK_SET, 0, 0, 0 };
7     fcntl(fd, F_SETLK, &lock);    /* blocking read lock */
8 }
9 void unlock_file(int fd) {
10    struct flock lock = { F_UNLCK, SEEK_SET, 0, 0, 0 };
11    fcntl(fd, F_SETLK, &lock);
12 }

```

Every file operation (reading voterlist.txt, appending voted.txt, updating tally.txt, appending audit.log) acquires the appropriate lock first, ensuring thread safety when multiple voter threads run concurrently.

2.8 Token-Based Authentication

Voter tokens are generated in auth.c using the **djb2** hash of the voter ID, the current Unix timestamp, and a secret key:

Listing 8: Token generation and validation

```

1 void generate_token(char *voter_id, char *token) {
2     long timestamp = time(NULL);
3     char data[100];
4     sprintf(data, "%s%ld%s", voter_id, timestamp, SECRET);
5     unsigned long sig = hash(data);
6     sprintf(token, "%s.%ld.%lu", voter_id, timestamp, sig);
7 }
8
9 int validate_token(char *token) {
10    char voter_id[20]; long timestamp; unsigned long sig;
11    sscanf(token, "%[^.].%ld.%lu", voter_id, &timestamp, &sig);
12    char data[100];
13    sprintf(data, "%s%ld%s", voter_id, timestamp, SECRET);
14    if (hash(data) != sig) return 0;    /* tampered */
15    if (time(NULL) - timestamp > EXPIRY) return 0;    /* expired */
16    return 1;
17 }

```

Tokens expire after 300 seconds (5 minutes).

2.9 Hash-Chained Audit Log

Every vote appends a new entry to `audit.log` that includes the hash of the previous entry, creating a tamper-evident chain:

Listing 9: Audit log entry format and chaining

```
1  /* Entry format: <no> | <prev_hash> | <voter_id> | <candidate> |
   * <hash> */
2  sprintf(data, "%s%s%c", last_hash, voter_id, candidate);
3  unsigned long curr_hash = hash(data);
4  fprintf(fp, "%d | %s | %s | %c | %lu\n",
5          entry_no, last_hash, voter_id, candidate, curr_hash);
```

Verification replays the entire chain, detecting: missing log file, entry number gaps, broken hash links, or tampered vote data.

3 Sample Output

3.1 Central Authority Startup and Menu

Listing 10: Authority startup and interactive menu

```
1  $ ./bin/authority
2  === Central Authority Authentication ===
3  Enter password: admin123
4  Authentication successful!
5
6
7  1. Enable   voting (all regions)
8  2. Disable voting (all regions)
9  3. Enable   voting (specific region)
10 4. Disable voting (specific region)
11 5. Verify   servers
12 6. Results
13 7. Exit
14 Choice:
```

3.2 Regional Server Startup

Listing 11: Starting Region R1 server

```
1  $ ./bin/server
2  Select region (R1, R2, or R3): R1
3  Voting is initially disabled
4  Region R1 server listening on port 8080
```

3.3 Enable Voting and Receiving a Vote

Listing 12: Authority enables voting; server receives a vote

```
1  Choice: 1
```

```
2 Voting enabled for 1 region(s)
3
4 --- server terminal ---
5 Voting enabled (SIGUSR1 received)
6 Received vote: A
```

3.4 Voter Client Session

Listing 13: A voter successfully casting a vote

```
1 $ ./bin/client
2 Select region (R1, R2, or R3): R1
3 Connected to region R1 server.
4
5 Role:
6     1. Voter
7     2. Admin
8 Choice: 1
9 Enter voter ID: 101
10 Token received.
11 Choose your candidate (A-Z): A
12 Vote status: SUCCESS
```

3.5 Duplicate Vote Rejection

Listing 14: Second vote attempt from the same voter is rejected

```
1 $ ./bin/client
2 Select region (R1, R2, or R3): R1
3 Connected to region R1 server.
4 Role: 1
5 Enter voter ID: 101
6 Token received.
7 Choose your candidate (A-Z): B
8 Vote status: FAILURE
```

3.6 Audit Verification

Listing 15: Audit verification from the Authority

```
1 Choice: 5
2 Verifying servers...
3 Region R1: audit verified successfully
4 Region R2: audit verified successfully
5 Region R3: audit verified successfully
6 Verification complete.
```

3.7 Result Collection with Integrity Check

Listing 16: Collecting results with global integrity check

```

1 Choice: 6
2 Disabling voting and collecting results...
3 Voting disabled for 3 region(s)
4
5 === RESULTS ===
6 Regions responded: 3/3
7   A: 5
8   B: 3
9   C: 2
10 Total votes cast: 10
11 Global unique voters: 10
12 Integrity check PASSED

```

3.8 Sample Audit Log

Listing 17: Contents of regional_server/data/R1/audit.log

```

1 1 | 0          | 101 | A | 229417091283719
2 2 | 229417091283719 | 102 | B | 874920183746251
3 3 | 874920183746251 | 103 | A | 192837465019283

```

4 Challenges Faced and Solutions

4.1 Signal Delivery to Multi-threaded Processes

Challenge: POSIX signals can be received by any thread in a multi-threaded process. When the Authority sends `SIGUSR1` to a Regional Server, the signal may reach a voting thread instead of the main thread, which can cause unexpected behaviour.

Solution: The signal handlers only update a volatile `sig_atomic_t` `vote_flag` variable. Voting threads access this variable using a `pthread_mutex`, ensuring safe reads and writes no matter which thread receives the signal. Using `sig_atomic_t` ensures the assignment happens safely as a single operation.

4.2 Race Condition on the Voted File

Challenge: Multiple voter threads may run the "check if voted, then append" process at the same time. Without locking, two threads with the same voter ID could both pass the check before either writes to the file, causing duplicate votes.

Solution: Before checking `voted.txt`, each thread acquires a `F_WRLCK` (write lock) using `fcntl`. The lock is kept during both the check and the append, making the full check-and-write process atomic across threads and processes.

4.3 IPC Message Routing with Multiple Servers

Challenge: All servers use the same two message queues. Without proper routing, a message meant for one server could be received by another.

Solution: The `type` field in `struct msg` is used as a filter for `msgrcv`. Messages from the Authority to a server use `type = server_pid`, while replies to the Authority use `type = 1`. This ensures each process only receives its own messages.

4.4 Shared Memory Slot Assignment for Result Collection

Challenge: When creating the result shared memory segment, the Authority must assign one slot to each server so every server knows where to store its result.

Solution: The Authority fills the `result_shm` entries with the PID of each registered server before sending the `RESULTS` command. Each server finds its matching PID and writes its tally there. A `ready` flag (0/1/-1) allows the Authority to wait until all servers respond or until the 5-second timeout is reached.

5 Conclusion

The Distributed Voting System successfully demonstrates the integration of several core systems programming concepts in a single coherent application. POSIX message queues and shared memory handle structured inter-process communication. UNIX signals provide lightweight real-time control of the voting window. TCP sockets and `pthread`s enable concurrent multi-client service. `fcntl` file locking ensures data integrity under concurrency. The hash-chained audit log provides tamper evidence, and the token-based authentication prevents impersonation and replay attacks.